

## Step 1: Requirements Gathering & System Architecture

Define the key components and workflows of your system.

- **User Roles & Access Levels:**
  - **Students:** Search existing projects, view project summaries/abstracts, submit proposals (if extended).
  - **Faculty / Department Heads:** Upload finalized project documents (PDFs), trigger similarity checks, approve/reject project titles.
  - **System Admin:** Manage university data, department lists, and user access.
- **Core Technical Pipeline:**
  1. **Document Ingestion:** Extract plain text from uploaded PDFs/Word docs.
  2. **Indexing & Database:** Store project metadata (Title, Abstract, Department, Year, University) and full text vectors.
  3. **Similarity & Plagiarism Engine:** Compare input text against existing documents in the database using Natural Language Processing (NLP) / vector search.

## Step 2: Designing the Similarity Search Engine

The heart of your project is **text comparison**.

### A. Semantic Search

*For Students searching titles/problem statements*

Instead of matching exact words, use **semantic embeddings** so that "Car Identification System" matches "Automotive License Plate Recognition."

- **Technique:** Generate vector embeddings using a pre-trained model (e.g., sentence-transformers/all-MiniLM-L6-v2 or multilingual models if dealing with Arabic text).
- **Database:** Use a vector database or vector search extension like **pgvector** (PostgreSQL) or **Qdrant / Faiss**.
- **Output:** Return top-matching project abstracts sorted by cosine similarity score.

### B. Text Matching & Plagiarism Detection

*For Department Heads and Academic staff*

For detecting direct copying or paraphrasing in full project reports:

- **Preprocessing:** Extract text from PDFs (using pdfplumber or pypdf), clean whitespaces, remove stop words.
- **Exact & N-Gram Matching:** Use algorithms like **TF-IDF + Cosine Similarity** or **Locality-Sensitive Hashing (LSH)** for fast document comparison.
- **String Matching / Diff Analysis:** Highlight exact matching sentences/passages

## **Step 3: Implementation**

1. **Database Schema & Data Collection**
2. **Text Extraction & Pipeline Development**
3. **Search & Plagiarism Algorithm Implementation**
4. **Web Dashboard & User Interface**
5. **System Testing & Validation**